arXiv:2604.20912v2 [quant-ph] 15 Jun 2026

# Quantum–HPC Software Stacks and the openQSE Reference Architecture: A Survey

Amir Shehata* [1], Brian Austin [2], Tom Beck [1], Lukas Burgholzer [3,4], Alex Chernoguzov [5], Spencer Churchill [6], Andrea Delgado [7], Yasuko Eckert[8], Jeffery Heckey[9], Kevin Kissell[10], Katherine Klymko [2], Josh Moles [6], Thomas Naughton [1], Lee James O'Riordan [11], Christian Ortiz Pauyac [12], Guen Prawiroatmodjo [6], Ermal Rrapaj [2], Emre Sahin [17], Jiri Schindler [6], Laura Schulz [13], Sebastian Stern[9], Tyler Takeshita [9], Miwako Tsuji [14,15], Oscar Wallis [17], Aleksander Wennersteen [16], Travis Humble [1], and Martin Schulz [4]

[1]Oak Ridge National Laboratory, Oak Ridge, TN, USA
[2]Lawrence Berkeley National Laboratory, Berkeley, CA, USA
[3]Munich Quantum Software Company, Munich, Germany
[4]Technical University of Munich, Munich, Germany
[5]Quantinuum, Broomfield, CO, USA
[6]IonQ, College Park, MD, USA
[7]Qblox, Oak Ridge, TN, USA
[8]AMD, Bellevue, WA, USA
[9]Amazon Web Services, Seattle, WA, USA
[10]Alice and Bob, Paris, France
[11]Xanadu Quantum Technologies Inc., Toronto, Canada
[12]Quantum Brilliance, Munich, Germany
[13]Argonne National Laboratory, Chicago, IL, USA
[14]RIKEN R-CCS, Kobe, Japan
[15]University of Tsukuba, Tsukuba, Japan
[16]Pasqal, Palaiseau, France
[17]The Hartree Centre, STFC, Warrington, UK

**Abstract**

Quantum resources are increasingly integrated into high-performance computing (HPC) and cloud environments, but quantum high-performance computing (QHPC) software stacks remain isolated, often proprietary, full-stack solutions lacking common interfaces across runtime, resource management, orchestration, and execution layers. This paper analyzes nine production QHPC stacks and identifies common design patterns and emerging requirements, covering deployment models, application interaction patterns, SDK support, and readiness for fault-tolerant operation. The survey exposes consistent needs in runtime abstraction, resource management, interconnect semantics, and observability. Based on these findings, we propose the open quantum-HPC software ecosystem (openQSE) reference architecture as a first step toward unifying the state-of-the-practice. openQSE defines a set of layer boundaries that allow different implementations to interoperate while preserving deployment flexibility, and is structured to support both current noisy intermediate-scale quantum (NISQ) workloads and future fault-tolerant quantum computing (FTQC) systems without changes to upper-layer application interfaces.

*Corresponding author: shehataa@ornl.gov

# 1 Overview

Quantum computers do not operate in isolation. Practical quantum execution depends on classical software and hardware for compilation, orchestration, control, scheduling, data movement, and error-management functions. As a result, integrating quantum resources into high-performance computing (HPC) environments is an end-to-end systems problem, not only a device-access problem. HPC centers are already deploying and evaluating these systems across multiple providers and deployment models [1, 2, 3, 4].

While large strides have been made in this area over the last few years, it has led to a situation where this integration currently occurs through many independent, fragmented software paths. Many interfaces remain provider-specific, assumptions vary between cloud and on-premises environments, and sites often rebuild similar integration logic for job submission, resource description, runtime interaction, and security handling. This fragmentation increases maintenance cost, slows comparative evaluations, and weakens portability for applications and operations teams.

In this paper we review the current state-of-the-practice in this area, analyzing the most prominent examples of quantum high-performance computing (QHPC) software stacks. We compare their architectures and design decisions, review key interfaces and user accesss modes, and characterize their current state. Based on that, we gather a comprehensive picture of the current QHPC approaches and offer a first-of-a-kind mapping into a single blueprint, which can then be used to drive a common community architecture. This forms the input for the open quantum-HPC software ecosystem (openQSE) initiative recently formed by major contributors, including HPC centers, vendors, and users of QHPC systems.

Within that context, this paper analyzes nine representative QHPC stacks, developed by industry and research. They span all major deployment options, i.e., managed cloud services, scheduler-centric datacenter integration, and brokered-offload designs. In particular, we include the QHPC stacks from AWS, IBM, IonQ, JHPC-Quantum, MQSS, Pasqal, Quantinuum, Quantum Brilliance, and Xanadu, and compare them based on their key properties. We explore convergence points and areas for future development in runtime boundaries, resource management, and interconnect/control assumptions. We then form a common reference architecture that is powerful enough to map all identified properties and mechanisms into a single architecture blueprint.

In summary, our paper makes three main contributions:

- **Quantum-HPC Stack Analysis**: We review the state-of-the-practice in QHPC software stacks by examining nine widely used representative frameworks. We identify common architectural patterns and important differences across current approaches and synthesize them into an interface convergence matrix (Table 1) that maps where the field is converging and where it is not.

- **Quantum-HPC Requirements Framework**: We introduce a requirements framework that spans deployment environments, hardware modalities, application patterns, and quantum-computing eras, providing a structured design space to consider current and future QHPC software systems.

- **openQSE Reference Architecture**: We propose the community-driven openQSE architecture as a vendor-neutral layered reference architecture that provides a holistic view of the integrated QHPC system without assigning ownership of specific layers to vendors or HPC centers. It preserves implementation flexibility, while stabilizing interfaces for portable, interoperable, and future-ready integration.

## 2 Problem Statement

Quantum computers are expected to become useful in hybrid scientific workflows for problem classes, such as Hamiltonian simulation, optimization, and quantum machine learning, with broader application areas including electronic structure and related chemistry, materials, and data-analysis workloads [5, 6]. In order for those workflows to become practical for scientists and facilities, though, quantum computers must be integrated with HPC resources in an intelligent and efficient manner. This integration will avoid repeatedly adapting to different provider-specific interfaces, runtime assumptions, and deployment models. Furthermore, it will remove operational overhead for data centers that need to compare evolving quantum platforms on a common basis and for scientists who need to evaluate multiple technologies.

Many groups and projects are working on this kind of integration, creating a broad field of individually grown solutions. As we mature the field, it will be critical to track the state-of-the-practice of current approaches, to gather differences and similarities, and to track the specific needs and requirements that drove individual design decisions. For such a state-of-the-practice survey, we must first establish a needed framework to understand the fundamentally different scaling models and allocation semantics that drive the use of quantum resources. HPC capacity scales by adding comparatively modular and independently scalable resources such as racks, nodes, cores, memory, and GPUs, which can be allocated incrementally or fractionally to users. By contrast, the capacity of quantum processing units (QPUs) scales through the addition and improvement of qubits, which remain scarce and tightly coupled. They cannot be scheduled efficiently as fractional resources. Quantum information cannot be copied, meaning that they also cannot be pre-empively scheduled or checkpointed. These distinctions are architecturally significant, as they shape scheduling, fairness, utilization, and the design of hybrid workflows.

## 3 Classification Axes

Any practical QHPC architecture must, therefore, account for both sides of the scaling asymmetry as well as handle a broad range of requirements in order to use classical and quantum resources efficiently. To enable a fair comparison in our analysis of the state-of-the-practice, we group requirements and solutions along four distinct axes of classification: where the system is deployed, which quantum-hardware modality is being integrated, how applications use it, and which quantum-computing era, noisy intermediate-scale quantum (NISQ) or fault-tolerant quantum computing (FTQC) (or both), it must support.

### 3.1 Axis 1: Deployment Environment

The two primary **deployment environments** for QHPC stacks are cloud and on-premises operation. Each imposes different integration, orchestration, latency, and ownership constraints, and a stack must support both without architectural rework as well as compositions of the two across administrative trust boundaries (often referred to as federated operation).

In **on-premises** deployments, the HPC center is responsible for integrating the quantum system into the local facility environment, including resource management, security policy, data movement, and user access paths [3, 1, 4]. This creates practical design questions beyond merely attaching a device. One issue is how the quantum resource is presented to users through the center's software stack. Another is the placement of classical compute beside the quantum resource to handle compilation, runtime services, orchestration, quantum error correction (QEC) and control. The

center must then decide whether those nearby classical resources are dedicated to quantum operation or shared with other workloads.

In the **cloud** model, quantum resources may be consumed from an HPC-center perspective as a form of capability extension. From the outside, the resource may appear to be an external endpoint. On the provider side, however, the environment still resembles a datacenter with its own orchestration stack, service boundaries, and classical execution tiers. In practice, these environments often rely on service-managed and containerized execution models rather than traditional HPC scheduler-native integration; for example, Amazon Braket Hybrid Jobs supports provider-managed classical instances and custom container images [7, 8].

**Federated operation** is best understood as a composition of the two primary models across multiple administrative trust boundaries rather than as a third independent category. The functions it depends on: authentication, authorization, scheduling, and data movement, are required in all deployments, but federated settings demand that they interoperate across sites under coordinated policy. Much of that coordination lies outside the stack's scope and is the responsibility of the participating operators; the stack's responsibility is to provide the hooks (pluggable identity providers, resource description portable across schedulers, secure data paths) that make such cross-domain operation possible.

### 3.2 Axis 2: Quantum Hardware Modality

Quantum computers can be built based on a wide variety of different implementations of quantum mechanical principles. This is usually referred to as its modality. Among the wide range of approaches that exist, Diamond Vacancies, Neutral/Cold Atoms, Photonic Systems, Spin Qubits, Superconducting Qubits, and Trapped Ions have to date received the most attention. These modalities impose different latency, control, and orchestration requirements, especially when considering QEC, whose operations must complete within the coherence time of the qubits. As a consequence, current software stacks typically target a single modality, or a small subset, to simplify optimization and control. However, long-term scalability will require well-defined abstraction layers that decouple higher-level software from hardware-specific details. A flexible tool and compiler pipeline capable of adapting transformations, lowering, and optimization to modality-specific constraints while preserving a consistent interface to upper layers is also required.

### 3.3 Axis 3: Application Interaction Patterns

The type of quantum workload shapes the requirements a QHPC software architecture must adhere to. For that, this paper identifies two meta-patterns, workflow-centric and accelerator-centric. Both are relevant across cloud and on-premises deployments.

#### 3.3.1 Workflow-Centric Pattern

In the workflow-centric pattern, applications consist of independent classical and quantum stages, each maintaining its own state, if any. They are separated by workflow boundaries and coordinated by a scheduler or workflow engine. Direct interaction between these stages from within the execution of a stage is explicitly excluded, as data/control exchange occurs only at stage boundaries rather than through tight runtime coupling. Typical workflow systems treat HPC and QPU resources equally, delegating work to whichever system is best suited. There is no requirement or implication that multiple workflow systems are used or that HPC needs to have a workflow system separate from the one coordinating quantum and classical hybrid workflow.

This pattern is well-suited for execution across sites, allowing quantum steps to be offloaded to remote resources, including cloud-based systems, without requiring low-latency coupling to the HPC stages. It also applies to on-prem configurations where the quantum stage remains a distinct part of a larger workflow.

#### 3.3.2 Accelerator-Centric Pattern

In the accelerator-centric pattern, applications are driven by a larger HPC program that invokes quantum kernels during execution, analogous to GPU accelerator offload. These kernels are typically fine-grained and dispatched on demand, with HPC and quantum resources co-allocated to enable low-latency interaction and efficient execution. This requires careful consideration to avoid HPC idling or timeout behavior

Unlike the workflow-centric model, this pattern requires tighter coupling between HPC and QPU, support for shared state across kernel invocations, and frequent synchronization or runtime feedback, including mid-circuit interaction when supported. Efficient utilization also motivates multi-tenant resource sharing through a second-level quantum scheduler, enabling independent kernel scheduling with explicit quality of service (QoS) and bounded-progress guarantees.

### 3.4 Axis 4: Targeted Quantum Computing Era

While current systems target NISQ workloads, future systems will support FTQC, with both eras expected to coexist for some time. Software stacks, therefore, differ in which era they support and how much of this distinction is exposed to user-facing layers. In the NISQ era, application execution often includes algorithm-specific hybrid loops (e.g., variational quantum eigensolver (VQE)) that are performance-sensitive but not strictly timing-critical. In FTQC, execution introduces inner QEC loops where syndrome readout and correction must operate under strict timing constraints and affect the correctness of execution.

This shift can be understood through explicit timing-domain separation. The physical timing domain (PTD) [9] captures waveform-level interactions with device physics, the deterministic timing domain (DTD) handles hardware-local control and fast feedback, and the real-time domain (RTD) supports bounded-latency classical processing, such as decoding and adaptive control. Higher software layers operate in the application time domain (ATD), relying on abstractions that isolate these timing constraints.

This separation motivates a runtime interface that abstracts application execution from hardware-specific details, allowing the same application model to span NISQ and FTQC systems. At the same time, lower-level hardware capabilities, such as conditional control, parameter streaming, and feedback primitives, should be exposed through stable, queryable APIs, enabling higher layers to make informed execution and compilation decisions without embedding device-specific logic. This balance allows the architecture to remain stable while accommodating evolving FTQC requirements.

## 4 First Established Cross-Stack Interfaces

Before turning to complete software stacks, we highlight two emerging interface community efforts that address recurring integration problems across the QHPC landscape: quantum device management interface (QDMI) and quantum resource management interface (QRMI).

Their emergence and uptake reflect a broader trend towards decoupled and interoperable interfaces.

### 4.1 Quantum Device Management Interface (QDMI)

QDMI [10] defines a C-level standard for interacting with quantum devices, providing a backend-agnostic interface for device discovery, capability queries, job lifecycle management, and result retrieval. It provides a device-facing abstraction boundary in QHPC systems, allowing upper layers to interact with quantum hardware without embedding vendor-specific APIs. In this role, QDMI exposes structured hardware information that compiler toolchains and runtime systems can use, while remaining independent of higher-level resource management policies.

QDMI is built around a layered lifecycle model supporting authenticated, multi-tenant device sessions. It exposes hardware properties at device, qubit, and operation levels, enabling hardware-aware compilation and informing scheduling decisions without requiring hardware-specific logic in upper layers. It also supports asynchronous execution tracking and low-latency interaction, allowing integration into both workflow-centric and accelerator-centric execution patterns while remaining focused on device interaction rather than resource allocation semantics.

### 4.2 Quantum Resource Management Interface (QRMI)

QRMI [1] is a Rust-based, vendor-neutral interface that provides Rust, C, and Python APIs for interacting with quantum resources. It abstracts resource acquisition, managed execution, resource selection, allocation, job submission, and lifecycle management—creating a resource-management boundary in QHPC systems. This allows upper layers to interact with quantum resources as schedulable entities without embedding site-specific or scheduler-specific control logic. By doing so, QRMI enables applications, schedulers, and resource managers to coordinate execution while remaining independent of specific device abstractions and compiler toolchains.

QRMI integrates with existing resource-management systems through dedicated plugin libraries, supporting Slurm, PBS, Open Cluster Scheduler, Flux, and IBM Spectrum Load Sharing Facility (LSF). By describing quantum resources using the same generic framework as other accelerators like GPUs, QRMI enables standard scheduling, allocation, cost management, and policy enforcement. This unified approach supports both workflow-centric and accelerator-centric execution patterns while maintaining consistent lifecycle semantics, and works seamlessly across cloud and on-premises QPU deployments. Rather than focusing on structured device abstraction, QRMI prioritizes resource management and execution. This design philosophy enables multiple front-ends (Qiskit, Pulser, Cuda-Q) to leverage QRMI without requiring framework-specific implementations. Additionally, QRMI enables job submission to simulators either through cloud connections or by piping hardware specifics to local simulators, demonstrating its adaptability across different execution environments without impacting workflow design.

## 5 Classifying and Mapping Existing QHPC Software Stacks

In this rapidly growing area of QHPC, a wide variety of software stacks have been developed independently by vendors and academic researchers alike. Prior surveys have examined this landscape from a literature perspective: Döbler and Jattana [11] categorize 107 publications spanning hybrid hardware architectures and software stacks, while Elsharkawy et al. [12] review quantum programming tools and their suitability for HPC integration. Our work is complementary: rather than surveying the literature, we examine production QHPC stacks as deployed today, and from that state-of-the-practice derive the common design patterns and interface boundaries that motivate a shared reference architecture. In order to understand their impact on the current state-of-the-practice and to map a path to a more unified ecosystem, we explore and classify nine

of the most common and available stacks, and derive commonalities and differences. For each stack we describe the architecture, its source and use cases, and classify them along the four axes introduced in Section 3. While this survey does not encompass the entire industry, it is sufficiently comprehensive to identify common design patterns which guide a shared architecture.

To further support the comparison, we map the workflow of each stack to a common hybrid job submission pattern, which we observed across all surveyed stacks and which is illustrated in Figure 1: a classical compute node submits a hybrid job through a Job Submission Abstraction layer, which decouples the application from the underlying resource manager. The Resource Management and Job Launch layer then allocates both quantum and classical resources concurrently or separately, depending on the user requirements, with the quantum resource encapsulating its own classical QPU support layer.

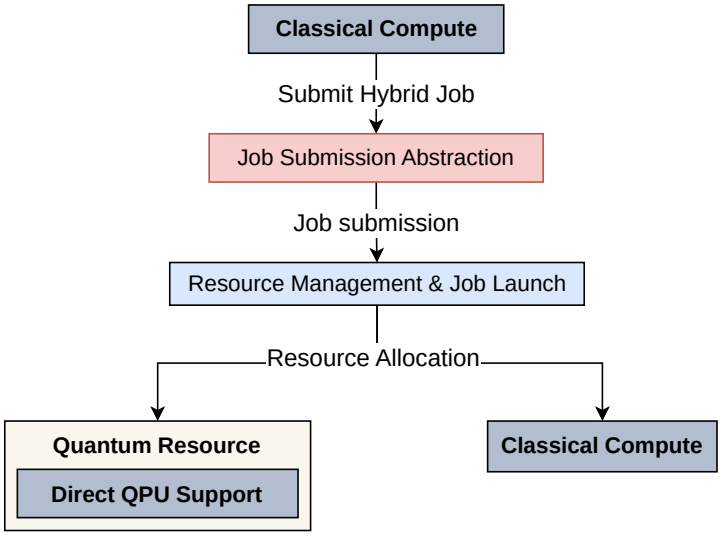


Figure 1: Common hybrid job submission workflow across all surveyed stacks. A Job Submission Abstraction decouples classical compute from the resource manager, which allocates quantum and classical resources in parallel. The quantum resource encapsulates its own classical QPU support layer, isolating hardware-specific control from upper stack layers.

In the following, we describe each of the nine stacks in our study, listed in alphabetical order and labeled based on the name of the respective developing company or project.

### 5.1 Stack 1: Amazon Web Services

Amazon Web Services (AWS) supports several QHPC deployment models: cloud, on-premises, federated, and mixed configurations. Cloud deployments range from single-service solutions using Amazon Braket, the AWS quantum computing service, to multi-service architectures combining Braket with HPC services. On-premises and federated deployments can integrate with Braket or leverage cloud HPC for on-premises quantum systems [13].

Amazon Braket provides unified access to QPU technologies from multiple hardware providers, including AQT, IonQ, IQM, QuEra, and Rigetti. The service accepts OpenQASM 3 [14] circuit representations (including the OpenPulse dialect for pulse-level access) and Analog Hamiltonian Simulation programs. The Amazon Braket SDK has open-source plugins for CUDA-Q, PennyLane, and Qiskit integration. Braket exposes device capability metadata including calibration and fidelity

data when available from QPU vendors, informing optional compiler optimization. Precompiled circuits can execute without optimization, and parametric circuits are cached for faster batch execution. Quantum circuits can be scheduled via on-demand access, priority access through Amazon Braket Hybrid Jobs, or exclusive access with reservations through Braket Direct. Fair-share queuing and HPC queue integration (e.g., Slurm plug-in architecture for node and job kontrol (SPANK) plugins) have not yet been implemented. Execution queues are resiliently maintained, and results persist in customer-owned Amazon S3 buckets.

Amazon Braket Hybrid Jobs supports single-service cloud QHPC by providing containerized execution environments that manage classical infrastructure lifecycle and quantum device integration. Multi-service and mixed deployments can leverage purpose-built HPC environments on Amazon EC2 clusters provisioned from more than 850 instance types [15] optimized to optimized for different computing requirements. Cluster infrastructure can be service-managed [16] or self-managed [17]. Cloud-native workflow orchestration [18] and batch scheduling [19] services provide alternatives with reduced operational overhead. These approaches support both workflow-centric and accelerator-centric patterns. Multi-service and mixed deployments allow the pairing of external QHPC software stacks with the Braket circuit execution model. Recently, the Amazon Braket implementation of the QDMI specification [10] enabled the integration of Braket into every QDMI-compatible QHPC software stack.

AWS supports federated cross-site access with external identity providers, fine-grained authorization with role-based and attribute-based access control, and protection of data at rest and in transit including VPN and AWS Direct Connect for private network connections.

### 5.2 Stack 2: IBM

IBM Quantum systems are based on superconducting qubit technology, which enables fast gate operations while imposing stringent requirements on timing, calibration, and control. Access to these systems is provided through multiple deployment models offered under the *IBM Quantum Platform*, including cloud-based access via the *Quantum Cloud API* and on-premises deployments accessed through the *Quantum System API*.

IBM uses the QRMI [1], which supports both APIs, and integrates QRMI with the Slurm workload manager via its SPANK plugin.

At the programming level, Qiskit serves as IBM's quantum SDK. Qiskit provides a Python interface as well as a C-API, which enables the use of additional programming environments, including C++ and Julia. In HPC-integrated deployments, Qiskit-based applications execute on classical HPC systems and access quantum systems via IBM's QRMI, allowing quantum execution to be incorporated into HPC workflows while keeping application logic decoupled from system-level resource management. Using this architecture, IBM has demonstrated both workflow-centric and accelerator-centric application patterns [20][21].

In the workflow-centric pattern, classical HPC jobs and quantum jobs are orchestrated by a workflow engine and executed as distinct stages within a larger pipeline, with loose temporal coupling between classical and quantum execution [21]. In contrast, the accelerator-centric pattern enables tighter integration between HPC and quantum systems, where classical and quantum executions are more closely coordinated and can partially overlap, improving overall system utilization and enabling closed-loop hybrid execution [20].

Taken together, these deployments correspond to the "quantum as a co-processor to HPC" model described in IBM's Quantum-Centric Supercomputing (QCSC) reference architecture [6]. While this model represents the dominant integration approach in current deployments, the QCSC reference architecture also outlines future phases with more tightly integrated execution models, including

near-time coupling and support for outer decoders.

### 5.3 Stack 3: IonQ

IonQ provides access to trapped-ion QPU resources through its *Quantum Platform* [22, 23], which spans scheduling, compilation, tenant management, and observability. It integrates with Qiskit [24], CUDA-Q [25], and PennyLane [26] SDKs. The primary deployment model is cloud-managed; on-premises and federated access pathways are emerging via QDMI [10] and QRMI [1] plugin implementations [2].

Scheduling is multi-tiered [2]: a platform tier manages tenant allocations via proportional fair-share; a QPU tier interleaves user jobs with calibrations, which consume accountable QPU time; and a hardware tier sequences low-level instructions. The control electronics layer surfaces timing information to upper tiers, and Section 8.4 discusses representative IonQ Forte gate-execution timings [27] and their cross-stack architectural implications.

Compilation applies device-agnostic optimization then device-specific lowering into the native gate set, guided by calibration data [2]. QRMI [1] exposes IonQ as a schedulable backend within HPC workload managers, while QDMI [10] provides the device-facing interface that the runtime layer uses to interact with IonQ hardware. This layering supports consistent operation against cloud-hosted IonQ deployments and could extend to on-premises IonQ deployments.

IonQ supports the workflow-centric pattern via direct API access, cloud marketplaces [22], and large-scale campaigns such as QC-AFQMC [13] coordinating IonQ QPUs with NVIDIA GPUs via Amazon Braket. FTQC support is on IonQ's roadmap, with the multi-tier control architecture providing the timing primitives needed for syndrome dispatch and measurement feedback in the deterministic and real-time domains.

### 5.4 Stack 4: JHPC-Quantum: Supercomputer Quantum Computer Computation (SQC) scheduler and ecosystem

The JHPC-Quantum Project [28], jointly developed by RIKEN, Softbank, the University of Tokyo, and University of Osaka targets an HPC-oriented integration of quantum and classical computing. Its primary objective is to enable efficient utilization of both HPC and QC resources, with particular emphasis on ensuring that large-scale HPC resources are not left idle or waiting on quantum processing. At the same time, standalone QC jobs are executed with lower priority, leveraging otherwise idle quantum resources to maximize overall system utilization. To this end, a lightweight scheduler called SQC scheduler is introduced between HPC and QC, designed to prioritize and promptly return results for requests originating from HPC workloads.

This scheduling mechanism not only orchestrates the flow of requests to quantum devices, but also abstracts away heterogeneity among backend quantum systems. Since users do not directly access individual QC backend servers, differences in their interfaces are effectively hidden. As a result, users are required to interact only with a unified scheduler interface, significantly simplifying the development and execution of hybrid quantum-HPC workflows.

By extending the execution layers of existing SDKs, such as Qiskit, TKET, and CUDA-Q to interface with the proposed scheduler, quantum programs written in these frameworks can also be executed without modification.

### 5.5 Stack 5: Munich Quantum Software Stack (MQSS)

The MQSS, developed as part of the Munich Quantum Valley (MQV), is a multi-layered architecture designed to integrate heterogeneous quantum hardware into HPC environments [29]. It was started

specifically as an open source solution with multi-modality support to match the developments in the MQV, but it also aims at broad applicability beyond MQV. As such, it supports both cloud-based and on-prem deployments, including tightly coupled integration within HPC systems and more loosely coupled remote-access configurations [30].

The architecture is organized into three layers. The front-end layer provides application-facing interfaces and supports frameworks such as Qiskit and PennyLane, domain-specific languages, as well as specific hybrid programming approaches, like QPI [31] or OpenMP integration [32]. The middle-end layer integrates quantum workflows into the HPC environment and is responsible for compilation, resource allocation, and orchestration of hybrid applications. It supports both workflow-centric and accelerator-centric execution patterns. This layer is supported by tooling from the Munich Quantum Toolkit (MQT) [33], including benchmarking (MQT Bench [34]), compilation across qubit modalities (MQT QMAP [35]), multi-level intermediate representation (MLIR)-based compilation infrastructure (MQT Core [36]), and scheduling and resource selection (MQT Predictor [37]). Integration with simple Linux utility for resource management (Slurm) is provided, including support for multi-resource scheduling through a second-level scheduler. The back-end layer interfaces with quantum hardware through open interfaces, particularly QDMI initially developed by the MQSS team [10], enabling interaction with multiple QPU backends.

MQSS has been deployed in an HPC setting at the Leibniz Supercomputing Centre, where it is used to manage a tightly coupled integration with a superconducting quantum device [3]. The stack exposes telemetry from both quantum resources and the surrounding HPC environment. It is currently used in NISQ-focused deployments, but includes architectural support for future FTQC operation, such as consideration of control electronics integration within the resource-management layer.

### 5.6 Stack 6: Pasqal

Pasqal develops and maintains a QHPC integration approach for its neutral-atom systems [38] that combines on-premises and cloud deployment support with explicit integration into HPC resource managers and a second device-level scheduling layer [39, 40, 41].

Pasqal relies on its QRMI [1] implementation to integrate with the HPC system scheduler and reuses it to provide framework-independent runtime access to its quantum resources, supporting its native Pulser [42], Qiskit primitives, and CUDA-Q [25] as programming frontends across on-premises integration, cloud bursting, and other mixed deployment modes. Pasqal supports both accelerator- and workflow-centric execution models.

Below the HPC resource manager, a second-level quantum-aware scheduler manages QPU-specific constraints that are difficult to capture directly in standard HPC schedulers, such as calibration-aware execution, and enforces center-specific usage policies. While the QPU itself must be allocated to a single active user or job at a time, this layer enables multiple user sessions at the access layer. It performs access control, manages execution windows, and can improve device utilization through mechanisms such as time-slicing and shot limits, while keeping the QPU's internal queue aligned with HPC scheduler policies such as priorities and maximum walltime.

Pasqal also exposes telemetry and observability capabilities through administrative interfaces in its on-premises stack, supporting system monitoring and operational visibility as described in [39].

### 5.7 Stack 7: Quantinuum

Quantinuum trapped-ion quantum computers are available via cloud access or via on-premise deployments. The QPUs are self-contained and capable of using any QEC code. The software stack

includes tools for application development, QPU programming, and compilation. Cloud hosted quantum computers are accessed via the Nexus [43] API while on-premise systems have an option for direct gRPC-based [44] access.

The programming language Guppy [45, 46] allows programmers to seamlessly combine classical and quantum logic in real-time with full support for measurement-based program flow control. Other development frameworks including Q# [47] and CUDA-Q [25] are supported via the industry standard quantum intermediate representation (QIR) [48] format. TKET optimizing compiler performs quantum optimizations on the intermediate representation and QPU targeting.

Quantinuum QPU scheduling is based on an adaptive time-sliced fair queuing approach. User job slices are scheduled just-in-time based on priority, order of submission, and previous execution history within a fairness window, alongside other scheduling attributes. Automated calibration routines are inserted between user job slices by the scheduler when necessary.

In HPC deployments, Quantinuum focuses on the workflow-centric pattern emphasizing complex interactions between HPC and QPU resources deployed side by side. Quantinuum workflow engine Tierkreis [49] is a task-based, asynchronous orchestrator for heterogeneous hardware types. It is programming language agnostic, allowing reuse of existing HPC software and tools.

The fundamental design principle is a worker-executor model. Workers provide atomic tasks to a workflow. Executors run assigned tasks on a target hardware, be it cloud, HPC or QPU. Currently, Tierkreis supports executors for Slurm, portable batch system (PBS) and pjsub on the HPC side; and Nexus and direct QPU submission on the quantum side. An example of a hybrid biochemistry application using Tierkreis combines existing HPC chemistry application with quantum kernels in a complex workflow [50].

### 5.8 Stack 8: Quantum Brilliance

Quantum Brilliance (QB) provides QHPC capabilities through its queue utility (QTIL) [51]. Implemented in Go (Golang), QTIL is responsible for quantum resource allocation and task scheduling, interfacing with the QPU web server via API calls.

Within the QB framework, QTIL integrates with the client SDK and the quantum development kit (QDK), where user-defined quantum circuits or pulse-level instructions are submitted via API. The QDK comprises the QPU and QCStack, a software stack that includes a web server and supporting libraries for functionalities such as API handling, transpilation, optimization, and calibration. In this context, QTIL operates as the resource management layer, orchestrating access to QPUs exposed through the QCStack interfaces.

Importantly, QTIL is not tightly coupled to the QB software stack. Its design allows it to interface with alternative client SDKs and backend systems, provided that compatible API endpoints are available. This makes QTIL adaptable to different software environments and hardware platforms.

QTIL manages resource allocation on a per-QPU basis, allowing users to specify parameters such as allocation start time, duration, and the number of QPUs required. QPU metadata is maintained in a relational SQL database (e.g., MySQL), enabling structured classification of resources based on configurable attributes. These attributes can be used to define logical groupings of QPUs and to enforce allocation policies.

Resource allocation is secured using JSON web tokens (JWT)s in combination with secret keys stored in the database. For integration into HPC environments, QTIL provides support for Slurm [52], enabling hybrid quantum-classical workflows. In contrast to approaches based on the SPANK plugin, QTIL adopts a license-based model using the Lua plugin. In this setup, users request classical resources (CPU/GPU) through Slurm as usual, while additionally specifying QPU requirements. This design minimizes changes to existing Slurm configurations, as modifications are

primarily limited to prolog and epilog scripts.

### 5.9 Stack 9: Xanadu

The Xanadu quantum stack [53] features PennyLane [26], a platform-agnostic frontend, and Catalyst [54], its dedicated compiler [2]. PennyLane maps high-level applications to structured hybrid programs, which Catalyst progressively lowers to device-specific operations for execution via a runtime layer. Catalyst compiles these workloads using an MLIR [55] toolchain, with classical and quantum instructions combined in a single optimized program. Through tools, like the Jeff framework [56], and a comprehensive plugin interface, the ecosystem natively interoperates with Qiskit [24], Cirq [57], Amazon Braket [7], and other vendor toolchains.

Xanadu has developed demonstrators of its quantum system technologies, including X8 [58], Borealis [59], and Aurora [60], while current development is focused on a utility-scale quantum computing system. Future access is expected to center on cloud and federated models delivered through remote PennyLane environments. Software deployment is highly flexible, with support for pre-built Python packages, customized Docker containers, Spack integration [61], and local source builds.

As Catalyst enables targeted heterogeneous compilation from a single user workload, both accelerator-centric and workflow-centric patterns are supported. A modular C++ runtime uses stubs for quantum instructions, tailoring execution to any target platform. This unified classical-quantum model natively supports local execution, remote submission, and directly integrated heterogeneous environments. Classical code components (control flow, memory semantics, linear algebra) compile directly for CPUs and accelerators (GPUs, TPUs) using MLIR/LLVM [62] backends and ecosystem libraries like JAX [63], ROCm™ [64], and CUDA [65].

While the software delegates workload-level scheduling and device provisioning to the surrounding system context, Catalyst provides the necessary tie-ins for these execution models. Furthermore, as Xanadu prepares for FTQC designs, Catalyst is actively integrating logical and physical layer quantum error correction passes into its lowering stack [66].

## 6 Design Patterns & Future Features

Across the current stack landscape, several design patterns emerge. The most consistent is a layered separation between applications, runtime services, compilation, and hardware-facing control. This structure allows adaptation across deployment environments without replacing the full stack, while enabling vendor specialization near the device behind more stable upper interfaces. Another pattern is the use of explicit runtime boundaries for submission, status tracking, and result return. Although not yet portable across stacks, these interfaces already simplify integration with workflows and facility software.

A staged view of program preparation is also common. Quantum programs are transformed and lowered through multiple compilation steps prior to execution, providing a natural insertion point for backend-specific optimization without exposing hardware details to applications. Reservation handling, scheduler integration, and capability discovery are increasingly treated as standard operational features. These capabilities enable coordination of shared resources, visibility into system state, and more predictable execution in hybrid environments.

Telemetry is similarly established across stacks. Systems routinely collect run-state information, operational health signals, execution metadata, and hardware-facing measurements for debugging, scheduling, and operations. While the specific data varies across modalities and vendors, its

consistent presence indicates that observability is an architectural requirement rather than an optional feature.

The next step is to consolidate these patterns into common interfaces. An application-facing runtime API would provide a stable offload contract and reduce dependence on stack-specific SDK entry points. A composable tool and pipeline model would allow compilation and transformation passes to be reused across frameworks instead of remaining isolated. Connection management requires similar abstraction, enabling transport and fabric differences to be handled without impacting upper layers. Communication semantics must also extend beyond one-way submission and result retrieval to support bidirectional interaction, allowing quantum-side services to trigger adaptation, event notifications, and resource requests.

Shared usage is already a common operating mode, driven by cloud providers that must support concurrent access from many users and therefore incorporate multi-tenant scheduling into their stacks. However, these mechanisms remain fragmented, and a unified approach to policy-aware sharing is needed as similar patterns extend into HPC environments. Calibration visibility is closely related, as maintenance and retuning directly affect scheduling decisions. Rather than enforcing a fixed telemetry model, a more durable goal is a standard mechanism for accessing system information that can evolve in a backward-compatible way as new hardware signals and control features emerge.

Finally, hardware capability exposure will become increasingly important as systems progress toward stronger error mitigation and fault-tolerant operation. Control-electronics features, hardware-near classical processing, and feedback paths should be surfaced through structured interfaces, so higher layers can make informed compilation, reservation, and execution decisions.

# 7 Discussion and Mapping to Common Blueprint

As shown in the previous section, the different stacks target different scenarios and support a wide range of properties and systems. Mapped to the classification axes, they do provide wide coverage across the entire design space. At the same time and despite their differences, they roughly map to our common layer model in Figure 2.

Table 1 reorients the mapping around *interfaces and components*: each row is an interface or software component, each column a stack, with cells indicating support (✓), either directly or via a translation layer, or absence of support (). Solid rows in Table 1 mark where the community is heading towards convergence (e.g., OpenQASM 3); sparse rows mark the boundaries that are still consolidating.

# 8 Introducing the openQSE Architecture

Our survey has shown the wide variety of current state-of-the-practice software solutions for various deployment scenarios, hardware modalities, interaction patterns, and targeted quantum era, as well as the different states in their development. At the same time it is also clear that not all stacks cover the entire scenario and that cross-stack support is sought after in many solutions. In particular, the establishment of the two interface approaches, QDMI and QRMI, shows a need to converge towards compatible approaches to further drive the ecosystem.

To accomplish this, we need a reference architecture, which allows us to map each stack to it, identify matching components, and extract interfaces that ultimately could lead us to community standards supporting interoperable stacks. Following the results of our survey, we propose openQSE, a community-driven architecture and specifications effort [2] to establish such a reference architecture. The goal is explicitly not to replace existing SDKs, vendor stacks, or open

| Interface / standard | AWS | IBM | IonQ | JHPC-Q | MQSS | Pasqal | QB | Qntm | Xan |
|---|---|---|---|---|---|---|---|---|---|
| *Deployment Model* | | | | | | | | | |
| Cloud | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | | ✓ | ✓ |
| On-premises | | ✓ | | ✓ | ✓ | ✓ | ✓ | ✓ | |
| *Resource-Level Scheduling* | | | | | | | | | |
| Slurm (SPANK / GRES / Lua) | ✓ | ✓ | | ✓ | ✓ | ✓ | ✓ | ✓ | |
| *SDK Interface* | | | | | | | | | |
| Qiskit [24] | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| PennyLane [26] | ✓ | ✓ | ✓ | ✓ | ✓ | | ✓ | ✓ | ✓ |
| TKET | | ✓ | ✓ | ✓ | ✓ | | ✓ | ✓ | ✓ |
| CUDA-Q [25] | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| *Quantum-Task Compiler / IR* | | | | | | | | | |
| MLIR-based lowering [55] | | | | | ✓ | | | | ✓ |
| Vendor-specific compiler | ✓ | ✓ | ✓ | | | ✓ | ✓ | ✓ | ✓ |
| OpenQASM [14] | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |
| QIR [48] | | | ✓ | | ✓ | | | ✓ | |
| *Resource / Device Interface* | | | | | | | | | |
| QDMI [10] (device, MQT) | ✓ | | ✓ | | ✓ | | | | |
| QRMI [1] (resource) | | ✓ | ✓ | | | ✓ | | | |
| *Quantum-Task Level Scheduling* | | | | | | | | | |
| *Stack-specific scheduling | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ | ✓ |

Table 1: Interface convergence across surveyed stacks (April 2026). ✓ indicates support (direct or via translation); blank indicates absence. All stacks expose vendor-defined REST/gRPC backends and support federated operation via external identity providers (rows omitted). *Stack-specific scheduling: a second-level scheduler beyond the per-device queue.

source efforts. Instead, the goal of this initiative is to define vendor-neutral interface boundaries so applications, runtimes, resource managers, and backend implementations can interoperate while still allowing implementation-specific or site-specific optimizations. This will enable the community to connect the different software stacks, exploit cross-stack functionality, and ultimately make all functionality available across the entire QHPC ecosystem.

As shown in Figure 2, the architecture is organized into five logical parts. These are an HPC control plane, one or more HPC/classical compute nodes, one or more quantum access nodes, one or more quantum resources, and shared infrastructure for network and data persistence. This decomposition illustrates a representative mapping of architectural responsibilities onto the heterogeneous compute elements typically found in HPC centers. While alternative mappings are possible, the configuration presented here is chosen to clearly separate concerns and align functional responsibilities with the capabilities of each component. Not all layers or compute elements are required in every deployment; the figure presents a comprehensive view of the architecture, with specific configurations determined by the target environment.

At the top of the stack, the HPC control node provides the logical control plane for job submission and resource abstraction. It may map to login nodes, workflow services, or control microservices rather than a single fixed system. The job submission abstraction layer is a standardized, system-agnostic representation of quantum resources and their capabilities. In practice, resource management systems (RMSs) (e.g., SLURM or Flux) each expose different mechanisms for describing and requesting resources; without a common model, this leads to fragmentation and non-portable job specifications. openQSE addresses this by defining a consistent resource description that decouples

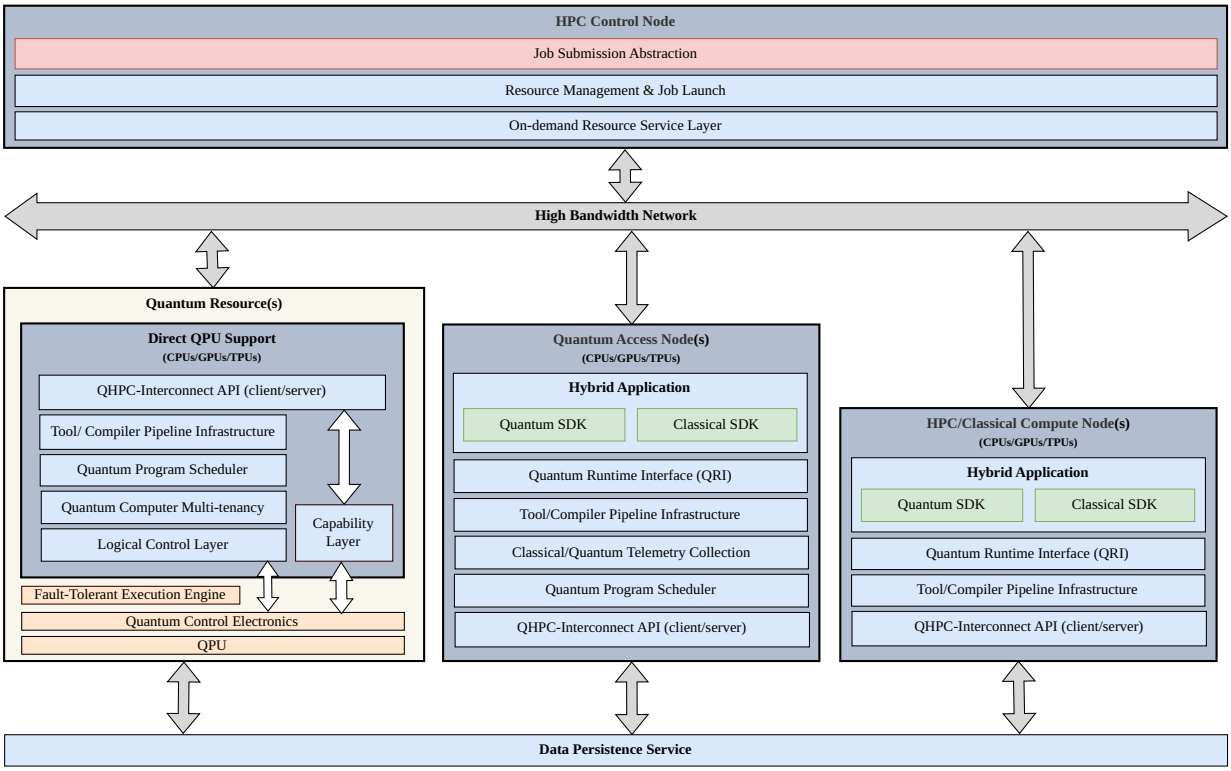


Figure 2: Proposed openQSE reference architecture for QHPC integration, showing a layered, deployment-agnostic stack with stable API contracts across control, access, and quantum-resource domains, including bidirectional QHPC interconnect, compiler/tool pipelines, scheduling/virtualization, and fault- tolerant execution paths. Not all layers may be needed in all deployments. For example, the Logical Control Layer would not be needed for a NISQ deployment.

job submission from any specific RMS, enabling the same job specification to be submitted across heterogeneous systems without modification. This also supports deployments in which quantum resources reside in different administrative domains. The on-demand resource service layer supports policy-bound expansion, allowing workflows to request temporary additional resources within declared burst envelopes.

The quantum resource defines the execution boundary for quantum workloads, spanning the QHPC-interconnect for quantum task submission down to physical pulse delivery. This boundary also includes the tool pipeline for resource-side lowering, the quantum program scheduler and multi-tenancy layers, and a logical control layer (LCL) that mediates access to hardware capabilities.

The scheduler and multi-tenancy layers manage how quantum workloads share a device while enforcing hardware constraints. In most current systems, only one circuit executes at a time, so these layers queue workloads according to defined policies. As hardware evolves to support concurrent execution across separate qubit regions, the same layers can be extended to schedule and manage parallel workloads.

The LCL operates in RTD. In the FTQC era, RTD represents the logical time of quantum operations. In both the NISQ and FTQC eras this layer exposes functionality such as parameter streaming, other device-specific features, such as incremental just-in-time (JIT) compilation, and support for hierarchical error correction codes. It may also expose capability discovery features, which may be statically configured or dynamically read from the device. The fault-tolerant execution

engine (FTEE) operates in the DTD. It consumes PTD execution state, makes bounded- latency adaptive decisions, and either issues corrective actions or escalates upward when the latency budget allows it. This interpretation aligns with recent QEC literature on active reset, conditional control, and dynamic circuits [67, 68, 69]. As a result, the degree of FTEE exposure to higher layers may vary across modalities. The architecture separates LCL and FTEE to make the intent explicit.

The quantum access node is the operational entry point for many workflows. It can host user-facing hybrid scripts or service-side preparation logic, and it can also act as a richer cluster tier with direct access to one or more quantum resources. HPC/classical compute nodes share most of the same runtime interfaces, while telemetry and shared operational services can be centralized outside the main execution path. Data persistence services and high-bandwidth networking support staging, classical checkpointing, and data movement across local and remote resources.

### 8.1 Quantum Runtime Interface

The quantum runtime interface (QRI) is the application-facing runtime boundary. It separates application logic from vendor-specific and site-specific execution details in a way that is similar in intent to HIP [64] for heterogeneous GPU environments. Applications can continue to use the preferred SDKs, but they interact with a common execution contract rather than a different backend-specific runtime for each target.

In openQSE, QRI is available wherever hybrid applications run, including HPC/classical nodes and quantum access nodes. It supports resource-aware submission, lifecycle management, and multi-resource launch at an abstraction level above low-level resource interfaces. This reduces direct application dependence on backend-specific interfaces and helps preserve portability when deployment environments change.

At the same time, current quantum hardware exhibits significant device-specific variability, and even the same system may change characteristics over time. While this limits the degree of uniformity that QRI can enforce, it remains a useful abstraction that can accommodate such variability through extensible interfaces and capability-aware execution models.

### 8.2 QHPC-Interconnect API

The QHPC-Interconnect API is a bidirectional interface that is the architectural boundary between different compute elements. It supports downstream and upstream invocation through an RPC layer, which allows different APIs to be implemented seamlessly, such as submit, configure, execute, retrieve operations, upstream signaling for events, callbacks, adaptation requests, and escalation triggers.

This interface domain also includes connection management, authentication, data compression and protection, and abstraction over the underlying high-speed fabric. These capabilities belong together because latency, trust, and transport behavior are coupled in real deployments. Bidirectional behavior matters when quantum-side services need upstream action, including additional classical processing, calibration notification or dynamic resource requests.

### 8.3 Compiler Tool Pipeline

The Tool/Compiler Pipeline Infrastructure is a governed model for combining algorithm-specific transformation tools and compiler passes in ordered stages. Pipelines are bound to inputs and workflows, so that a given path applies a consistent set of transformations. This supports reproducibility, policy enforcement, and performance tuning while still leaving room for domain-specific extensions.

Pipeline placement remains flexible. Some stages can run on HPC/classical nodes, while later backend-specific lowering can run on access nodes or near the quantum resource. The architecture does not force one compilation location. It standardizes how stages are described, connected, and validated. That makes it easier to reuse transformations across frameworks and gives ecosystem efforts, such as Jeff [56, 70], a clearer architectural target.

### 8.4 Control Electronics

Control electronics sit at the boundary between the PTD and the DTD. They translate bounded-latency digital commands into physical qubit operations, including waveform generation, pulse modulation, readout, and real-time feedback. Their behavior affects scheduling, throughput, and FTEE design, so they should be treated as an architectural concern rather than a purely vendor-internal detail.

IonQ makes this boundary especially visible. In the Forte system, an embedded controller applies RF pulse sequences for coherent gate execution, while the runtime maintains pulse parameters through recurring calibration cycles between circuit batches. Those cycles consume scheduler-visible QPU time. Reported measurements indicate that calibration and testing consume about 47% of total operational time, leaving 53% for user circuit execution. IonQ addresses this with a nanosecond-precision real-time tier and an upper-layer performance model that uses native gate parameters and hardware state for scheduling and pipelining [2]. On IonQ Forte, single-qubit gates are reported at 110–130 $\mu$s, two-qubit gates at 900–950 $\mu$s, and ion-chain cooling at about 3 ms per shot [27].

More generally, control electronics are moving beyond fixed waveform delivery. Parameter streaming, conditional branching, and feedback paths are becoming architecturally relevant capabilities. openQSE therefore needs a capability-discovery interface so upper layers can query supported timing windows, branching primitives, and feedback latencies instead of assuming the same path on every platform. That contract also needs to remain stable enough for QEC-driven corrective actions to be dispatched without redesigning the surrounding software stack.

## 9 Conclusion and Future Work

This paper examined nine representative QHPC software stacks along four classification axes to highlight areas of convergence as well as ongoing divergence. Current systems already offer useful building blocks for hybrid execution, but they differ in key aspects. These include how runtime boundaries are defined, how interconnect semantics is handled, what level of observability is provided, and what hardware capabilities are exposed to higher software layers. We proposed the openQSE architecture, a composable layer model for operating across cloud and on-premises environments.

This work has three main limitations. First, the analysis is based on publicly documented capabilities and vendor-described stack descriptions as of April 2026; the QHPC landscape is evolving rapidly and specific stack characteristics may shift. Second, the openQSE architecture is presented as a conceptual reference model rather than a reference implementation, and its concrete interface contracts remain to be specified. Third, no cross-stack quantitative performance comparison is provided; structural rather than measurement-based characterization is the focus of this survey.

Future work will include a structured study to evaluate each stack by collecting a wide range of metrics, such as execution latency, throughput, resource utilization, and integration overhead. This will provide insight into the best implementation for specific features and reveal where designs may be improved. Developing the openQSE reference implementation will also provide a practical environment to integrate components and expose interoperability issues early. This creates a

feedback loop where architectural decisions are validated in code. Findings can then guide iterative refinement and be shared with the community through contributions to open-source projects.

## Acknowledgments

The authors acknowledge the contributions of IBM for their insights on QHPC software stack design, as well as their participation in discussions and feedback that helped shape the perspectives presented in this paper.

The authors used Claude (Anthropic), Gemini (Google), Microsoft Copilot, and ChatGPT and Codex (OpenAI) for editorial assistance. Final document was reviewed and verified by the authors, who are solely responsible for the technical content.

## References

[1] U. Bacher *et al.*, "Quantum resources in resource management systems," arXiv:2506.10052, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2506.10052

[2] openQSE Initiative, "openQSE workshops (architecture presentations and stack evaluations)," GitHub repository. [Online]. Available: https://github.com/openQSE/Workshops, 2026, accessed: Mar. 3, 2026.

[3] E. Mansfield, S. Seegerer, P. Vesanen, J. Echavarria, M. N. Farooqi, B. Mete, and L. Schulz, "First practical experiences integrating quantum computers with hpc resources: A case study with a 20-qubit superconducting quantum computer," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops '25. New York, NY, USA: Association for Computing Machinery, 2025, pp. 1842–1850. [Online]. Available: https://doi.org/10.1145/3731599.3767551

[4] T. Häner, D. S. Steiger, J. Heinsoo *et al.*, "Euro-q-exa: Integrating quantum computers into hpc systems," *arXiv preprint arXiv:2509.12949*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2509.12949

[5] D. Camps, E. Rrapaj, K. Klymko, H. Kim, K. Gott, S. Darbha, J. Balewski, B. Austin, and N. J. Wright, "Quantum computing technology roadmaps and capability assessment for scientific computing–an analysis of use cases from the nersc workload," *arXiv preprint arXiv:2509.09882*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2509.09882

[6] S. Seelam, J. M. Chow, A. D. Corcoles, S. Sheldon, T. Mittal, A. Kandala, S. Dague, I. Hincks, H. Horii, B. Johnson, M. Le, H. Jamjoom, and J. M. Gambetta, "Reference architecture of a quantum-centric supercomputer," 2026, arXiv:2603.10970. [Online]. Available: https://doi.org/10.48550/arXiv.2603.10970

[7] Amazon Braket Developer Guide, "Working with Amazon Braket hybrid jobs," [Online]. Available: https://docs.aws.amazon.com/braket/latest/developerguide/braket-build-jobs.html, 2025, accessed: Mar. 3, 2026.

[8] ——, "Running hybrid jobs during a reservation," [Online]. Available: https://docs.aws.amazon.com/braket/latest/developerguide/braket-run-hybrid-jobs-with-reservation.html, 2025, accessed: Mar. 3, 2026.

[9] S. A. Caldwell *et al.*, "Platform architecture for tight coupling of high-performance computing with quantum processors," arXiv:2510.25213, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.25213

[10] R. Wille, L. Schmid, Y. Stade, J. Echavarria, M. Schulz, L. Schulz, and L. Burgholzer, "QDMI – quantum device management interface: Hardware-software interface for the Munich Quantum Software Stack," in *Proc. IEEE Int'l Conf. on Quantum Computing and Engineering (QCE)*, 2024, pp. 573–574. [Online]. Available: https://doi.org/10.1109/QCE60285.2024.10411

[11] P. Döbler and M. S. Jattana, "A survey on integrating quantum computers into high performance computing systems," 2025. [Online]. Available: https://arxiv.org/abs/2507.03540

[12] A. Elsharkawy, X.-T. M. To, P. Seitz, Y. Chen, Y. Stade, M. Geiger, Q. Huang, X. Guo, M. A. Ansari, C. B. Mendl, D. Kranzlmüller, and M. Schulz, "Integration of quantum accelerators with high performance computing—a review of quantum programming tools," *ACM Transactions on Quantum Computing*, vol. 6, no. 3, Jul. 2025. [Online]. Available: https://doi.org/10.1145/3743149

[13] L. Zhao *et al.*, "Quantum-classical auxiliary field quantum monte carlo with matchgate shadows on trapped ion quantum computers," arXiv:2506.22408, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2506.22408

[14] A. Cross, A. Javadi-Abhari, T. Alexander, N. De Beaudrap, L. S. Bishop, S. Heidel, C. A. Ryan, P. Sivarajah, J. Smolin, J. M. Gambetta *et al.*, "Openqasm 3: A broader and deeper quantum assembly language," *ACM Transactions on Quantum Computing*, vol. 3, no. 3, pp. 1–50, 2022. [Online]. Available: https://doi.org/10.1145/3505636

[15] Amazon Web Services, "Amazon EC2," [Online]. Available: https://aws.amazon.com/ec2/, 2026, accessed: Mar. 30, 2026.

[16] ——, "AWS Parallel Computing Service," [Online]. Available: https://aws.amazon.com/pcs/, 2026, accessed: Mar. 30, 2026.

[17] ——, "AWS ParallelCluster," [Online]. Available: https://aws.amazon.com/hpc/parallelcluster/, 2026, accessed: Mar. 30, 2026.

[18] ——, "AWS Step Functions - workflow orchestration," [Online]. Available: https://aws.amazon.com/step-functions/, 2026, accessed: Mar. 30, 2026.

[19] ——, "AWS Batch," [Online]. Available: https://aws.amazon.com/batch/, 2026, accessed: Mar. 30, 2026.

[20] T. Shirakawa, J. Robledo-Moreno, T. Itoko, V. Tripathi, K. Ueda, Y. Kawashima, L. Broers, W. Kirby, H. Pathak, H. Paik, M. Tsuji, Y. Kodama, M. Sato, C. Evangelinos, S. Seelam, R. Walkup, S. Yunoki, M. Motta, P. Jurcevic, H. Horii, and A. Mezzacapo, "Closed-loop calculations of electronic structure on a quantum processor and a classical supercomputer at full scale," 2025, arXiv:2511.00224. [Online]. Available: https://doi.org/10.48550/arXiv.2511.00224

[21] N. Kanazawa, Y. Morohoshi, H. Takahashi, Y. Kawashima, H. Horii, and K. Nakajima, "Observability architecture for quantum-centric supercomputing workflows," 2025, arXiv:2512.05484. [Online]. Available: https://doi.org/10.48550/arXiv.2512.05484

[22] IonQ, Inc., "IonQ quantum cloud," [Online]. Available: https://www.ionq.com/quantum-cloud, 2026, accessed: Mar. 23, 2026.

[23] openQSE Working Group, "openqse working group: Quantum computing software ecosystem workshop notes," https://github.com/openQSE/Workshops/blob/main/OpenQSE-WG/openQSE-IonQ-20260309.pdf, 2026, accessed: 2026-04-04.

[24] Qiskit contributors, "Qiskit: An open-source framework for quantum computing," [Online]. Available: https://github.com/Qiskit/qiskit, 2024. [Online]. Available: https://doi.org/10.5281/zenodo.2573505

[25] NVIDIA Corporation, "CUDA-Q: A platform for hybrid quantum-classical computing," [Online]. Available: https://developer.nvidia.com/cuda-q, 2026, accessed: Mar. 23, 2026.

[26] V. Bergholm *et al.*, "Pennylane: Automatic differentiation of hybrid quantum-classical computations," 2022. [Online]. Available: https://doi.org/10.48550/arXiv.1811.04968

[27] J.-S. Chen *et al.*, "Benchmarking a trapped-ion quantum computer with 30 qubits," *Quantum*, vol. 8, p. 1516, 2024. [Online]. Available: https://doi.org/10.22331/q-2024-11-07-1516

[28] openQSE Working Group, "openqse working group: Quantum computing software ecosystem workshop notes," https://github.com/openQSE/Workshops/blob/main/OpenQSE-WG/openQSE-RIKAN-20260113.pdf, 2026, accessed: 2026-04-04.

[29] L. Burgholzer, J. Echavarria, P. Hopf, Y. Stade, D. Rovara, L. Schmid, E. Kaya, B. Mete, M. N. Farooqi, M. Chung, M. De Pascale, L. Schulz, M. Schulz, and R. Wille, "The Munich Quantum Software Stack: Connecting End Users, Integrating Diverse Quantum Technologies, Accelerating HPC," in *SC/HPC Asia 2026*. ACM, 2026, pp. 55–67. [Online]. Available: https://doi.org/10.1145/3773656.3773669

[30] openQSE Working Group, "openqse working group: Quantum computing software ecosystem workshop notes," https://github.com/openQSE/Workshops/blob/main/OpenQSE-WG/openQSE-MQSC-20260105.pdf, 2026, accessed: 2026-04-04.

[31] E. Kaya, B. Mete, L. Schulz, M. N. Farooqi, J. Echavarria, and M. Schulz, "Qpi: A programming interface for quantum computers," in *2024 IEEE International Conference on Quantum Computing and Engineering (QCE)*, vol. 02, 2024, pp. 286–291. [Online]. Available: https://doi.org/10.1109/QCE60285.2024.10293

[32] J. K. Lee, O. T. Brown, M. Bull, M. Ruefenacht, J. Doerfert, M. Klemm, and M. Schulz, "Quantum task offloading with the openmp api," in *Posters at SC23*, 2023. [Online]. Available: https://sc23.supercomputing.org/proceedings/tech_poster/poster_files/rpost177s3-file3.pdf

[33] R. Wille, L. Berent, T. Forster, J. Kunasaikaran, K. Mato, T. Peham, N. Quetschlich, D. Rovara, A. Sander, L. Schmid, D. Schoenberger, Y. Stade, and L. Burgholzer, "The MQT handbook: A summary of design automation tools and software for quantum computing," in *Int'l Conf. on Quantum Software*, 2024. [Online]. Available: https://doi.org/10.1109/QSW62656.2024.00013

[34] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Bench: Benchmarking Software and Design Automation Tools for Quantum Computing," *Quantum*, vol. 7, p. 1062, 2023, MQT Bench is available at https://mqt-bench.app/. [Online]. Available: https://doi.org/10.22331/q-2023-07-20-1062

[35] R. Wille and L. Burgholzer, "MQT QMAP: Efficient quantum circuit mapping," in *International Symp. on Physical Design*, 2023. [Online]. Available: https://doi.org/10.1145/3569052.3578928

[36] L. Burgholzer, Y. Stade, T. Peham, and R. Wille, "MQT Core: The backbone of the Munich Quantum Toolkit (MQT)," *Journal of Open Source Software*, vol. 10, no. 108, p. 7478, 2025. [Online]. Available: https://doi.org/10.21105/joss.07478

[37] N. Quetschlich, L. Burgholzer, and R. Wille, "MQT Predictor: Automatic Device Selection with Device-Specific Circuit Compilation for Quantum Computing," *ACM Transactions on Quantum Computing (TQC)*, 2025. [Online]. Available: https://doi.org/10.1145/3673241

[38] L. Henriet, L. Beguin, A. Signoles, T. Lahaye, A. Browaeys, G.-O. Reymond, and C. Jurczak, "Quantum computing with neutral atoms," *Quantum*, vol. 4, p. 327, Sep. 2020. [Online]. Available: https://doi.org/10.22331/q-2020-09-21-327

[39] A. Wennersteen, M. Moreau, A. Nober, and M. Beji, "Towards a user-centric HPC-QC environment," in *Proc. SC '25 Workshops*, 2025. [Online]. Available: https://doi.org/10.1145/3731599.3767549

[40] openQSE Working Group, "openqse working group: Quantum computing software ecosystem workshop notes," https://github.com/openQSE/Workshops/blob/main/OpenQSE-WG/openQSE-Pasqal-20260119.pdf, 2026, accessed: 2026-04-04.

[41] A. Wennersteen, K. Bidzhiev, M. D'Arcangelo, M. Moreau, A. Quelle, A. Dauphin, and M. Beji, "Hybrid quantum classical algorithms: A cloud on-demand viewpoint," in *Quantum Engineering Sciences and Technologies for Industry and Services*, F. Barbaresco and F. Gerin, Eds. Cham: Springer Nature Switzerland, 2026, pp. 117–125. [Online]. Available: https://doi.org/10.1007/978-3-032-13855-2_11

[42] H. Silvério, S. Grijalva, C. Dalyac, L. Leclerc, P. J. Karalekas, N. Shammah, M. Beji, L.-P. Henry, and L. Henriet, "Pulser: An open-source package for the design of pulse sequences in programmable neutral-atom arrays," *Quantum*, vol. 6, p. 629, Jan. 2022. [Online]. Available: https://doi.org/10.22331/q-2022-01-24-629

[43] Quantinuum, "Quantinuum nexus," 2024. [Online]. Available: https://nexus.quantinuum.com/

[44] gRPC Authors, "grpc: A high performance, open source universal rpc framework," https://grpc.io/, 2024, accessed: 2024-05-28.

[45] M. Koch, A. Lawrence, K. Singhal, S. Sivarajah, and R. Duncan, "Guppy: Pythonic quantum-classical programming," in *Informal Proceedings of the Fourth International Workshop on Programming Languages for Quantum Computing (PLanQC '24)*, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2510.12582

[46] M. Koch, A. Borgna, C. Roy, A. Lawrence, K. Singhal, S. Sivarajah, and R. Duncan, "Imperative quantum programming with ownership and borrowing in guppy," in *Informal Proceedings of the Fifth International Workshop on Programming Languages for Quantum Computing (PLanQC '25)*, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.13082

[47] K. Svore, A. Geller, M. Troyer, J. Azariah, C. Granade, B. Heim, V. Kliuchnikov, M. Mykhailova, A. Paz, and M. Roetteler, "Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL," in *Proceedings of the Real World Domain*

*Specific Languages Workshop 2018*, ser. RWDSL2018. ACM, Feb. 2018. [Online]. Available: http://dx.doi.org/10.1145/3183895.3183901

[48] *QIR Specification*, QIR Alliance: https://qir-alliance.org. [Online]. Available: https://github.com/qir-alliance/qir-spec

[49] S. Sivarajah, L. Heidemann, A. Lawrence, and R. Duncan, "Tierkreis: a dataflow framework for hybrid quantum-classical computing," in *2022 IEEE/ACM Third International Workshop on Quantum Computing Software (QCS)*, 2022, pp. 12–21. [Online]. Available: https://doi.org/10.1109/QCS56647.2022.00007

[50] K. Yamamoto, R. Masui, T. Nakajima, M. Tsuji, M. Sato, P. Schow, L. Heidemann, M. Burke, P. Seitz, O. J. Backhouse, J. W. Pedersen, J. Children, C. Holliman, N. Lysne, D. Okuno, S. Sivarajah, D. M. Ramo, A. Chernoguzov, and R. Duncan, "Quantum-hpc hybrid computation of biomolecular excited-state energies," 2026. [Online]. Available: https://doi.org/10.48550/arXiv.2601.15677

[51] openQSE Working Group, "openqse working group: Quantum computing software ecosystem workshop notes," https://github.com/openQSE/Workshops/blob/main/OpenQSE-WG/openqSE-QB-20260219.pdf, 2026, accessed: 2026-04-04.

[52] Slurm Workload Manager Documentation, "scontrol (reservation and TRES semantics)," [Online]. Available: https://slurm.schedmd.com/scontrol.html, 2025, accessed: Mar. 3, 2026.

[53] openQSE Working Group, "openqse working group: Quantum computing software ecosystem workshop notes," https://github.com/openQSE/Workshops/blob/main/OpenQSE-WG/openQSE-Xanadu-20260303.pdf, 2026, accessed: 2026-04-04.

[54] D. Ittah, A. Asadi, E. O. Lopez, S. Mironov, S. Banning, R. Moyard, M. J. Peng, and J. Izaac, "Catalyst: a python jit compiler for auto-differentiable hybrid quantum programs," *Journal of Open Source Software*, vol. 9, no. 99, p. 6720, 2024. [Online]. Available: https://doi.org/10.21105/joss.06720

[55] C. Lattner *et al.*, "MLIR: A compiler infrastructure for the end of Moore's Law," arXiv:2002.11054, 2020. [Online]. Available: https://doi.org/10.48550/arXiv.2002.11054

[56] D. Ittah, "jeff: bridging quantum compilers," Talk abstract, Quantum Software 2.1 Workshop. [Online]. Available: https://sites.google.com/view/quantum-software-workshop-2025, 2025, accessed: Mar. 6, 2026.

[57] Google Quantum AI, "Cirq: A Python Framework for Creating, Editing, and Invoking Noisy Intermediate Scale Quantum (NISQ) Circuits," https://quantumai.google/cirq, 2024, accessed: 2026-04-03.

[58] J. M. Arrazola, V. Bergholm, K. Brádler, T. R. Bromley, M. J. Collins, I. Dhand, A. Fumagalli, T. Gerrits, A. Goussev, L. G. Helt *et al.*, "Quantum circuits with many photons on a programmable nanophotonic chip," *Nature*, vol. 591, no. 7848, pp. 54–60, 2021. [Online]. Available: https://doi.org/10.1038/s41586-021-03202-1

[59] L. S. Madsen, F. Laudenbach, M. F. Askarani, F. Rortais, T. Vincent, J. F. F. Bulmer, F. M. Miatto, L. Neuhaus, L. G. Helt, M. J. Collins, A. E. Lita, T. Gerrits, S. W. Nam, V. D. Vaidya, M. Menotti, I. Dhand, Z. Vernon, N. Quesada, and J. Lavoie, "Quantum computational

advantage with a programmable photonic processor," *Nature*, vol. 606, no. 7912, pp. 75–81, Jun. 2022. [Online]. Available: https://doi.org/10.1038/s41586-022-04725-x

[60] J. E. Bourassa, R. S. Chadwick, and et al., "Scaling and networking a modular photonic quantum computer," *Nature*, vol. 638, pp. 912–919, Jan 2025. [Online]. Available: https://doi.org/10.1038/s41586-024-08406-9

[61] T. Gamblin, M. LeGendre, M. R. Collette, G. L. Lee, A. Moody, B. R. de Supinski, and S. Futral, "The spack package manager: bringing order to hpc software chaos," in *SC '15: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, 2015, pp. 1–12. [Online]. Available: https://doi.org/10.1145/2807591.2807623

[62] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," in *Proceedings of the International Symposium on Code Generation and Optimization (CGO)*. IEEE, 2004, pp. 75–86. [Online]. Available: https://doi.org/10.1109/CGO.2004.1281665

[63] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[64] AMD ROCm Documentation, "What is HIP?" HIP 7.2 documentation. [Online]. Available: https://rocm.docs.amd.com/projects/HIP/en/develop/what_is_hip.html, 2025, accessed: Mar. 3, 2026.

[65] NVIDIA Corporation, "NVIDIA CUDA Parallel Computing Platform," https://developer.nvidia.com/cuda-zone, 2024, accessed: 2026-04-03.

[66] Xanadu, "Catalyst quantum compilation dialects," GitHub repository. [Online]. Available: https://github.com/PennyLaneAI/catalyst/tree/main/mlir/lib, 2026, accessed: Mar. 23, 2026.

[67] Quantum Machines Documentation, "Real-time feedback and active reset with OPX and QOP," [Online]. Available: https://docs.quantum-machines.co/1.2.3/docs/Introduction/use_case/active_reset/, 2026, accessed: Mar. 6, 2026.

[68] Qblox Documentation, "Conditional operations (QTM/QCM-RF II)," [Online]. Available: https://docs.qblox.com/en/main/cluster/q1_sequence_processor.html, 2026, accessed: Mar. 6, 2026.

[69] IBM Quantum Documentation, "Hardware considerations and limitations for dynamic circuits," [Online]. Available: https://docs.quantum.ibm.com/guides/dynamic-circuits-considerations, 2025, accessed: Mar. 6, 2026.

[70] Unitary Foundation, "Annual report 2025 (JEFF section, collaboration with Xanadu and Quantinuum)," [Online]. Available: https://willzeng.com/shared/UF_Annual_Report_2025.pdf, Jan. 2026, accessed: Mar. 6, 2026.